Robot Information Publishing

1. Course Content

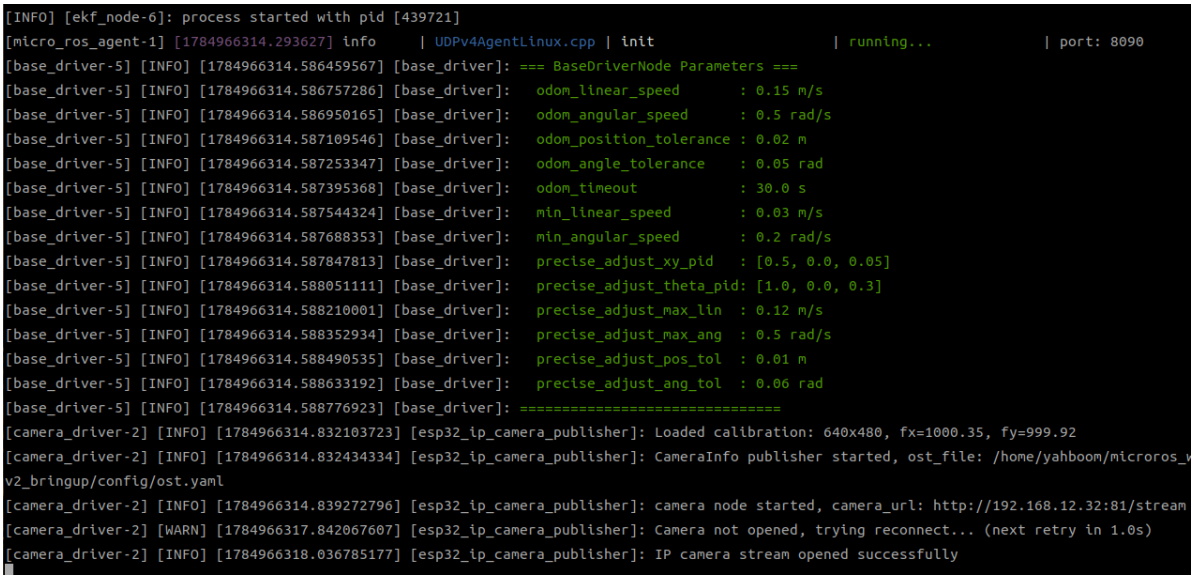
- Connect to the robot car through the micro-ROS agent, view topic data, and control the robot through topic and service interfaces

Course Objectives

- Understand how the car's sensor data enters the ROS 2 system
- Master the basic operations of subscribing to topics and publishing control commands through the command line

2. Preparation

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Demonstration

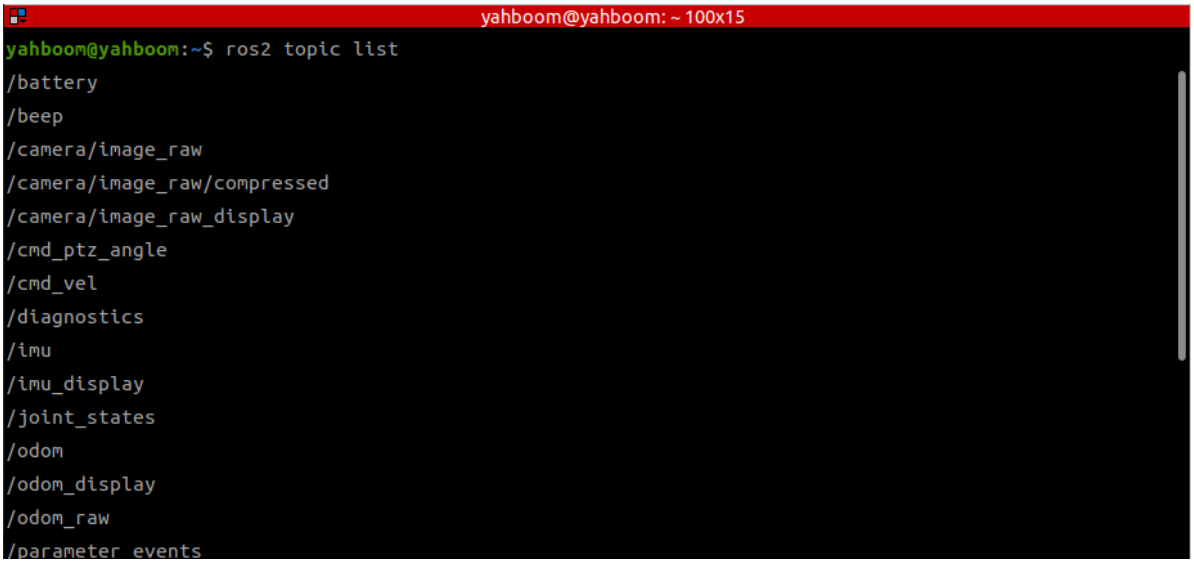
3.1 Chassis Sensor Topics

- Topic summary

Topic	Message Type	Description
`imu`	`sensor_msgs/Imu`	IMU angular velocity + linear acceleration
`odom_raw`	`nav_msgs/Odometry`	Accumulated wheel odometry
`scan`	`sensor_msgs/LaserScan`	LiDAR scan data
`battery`	`std_msgs/Float32`	Battery voltage
`ptz_angle`	`geometry_msgs/Vector3`	Current gimbal angles (x=S1 horizontal, y=S2 pitch, in degrees)

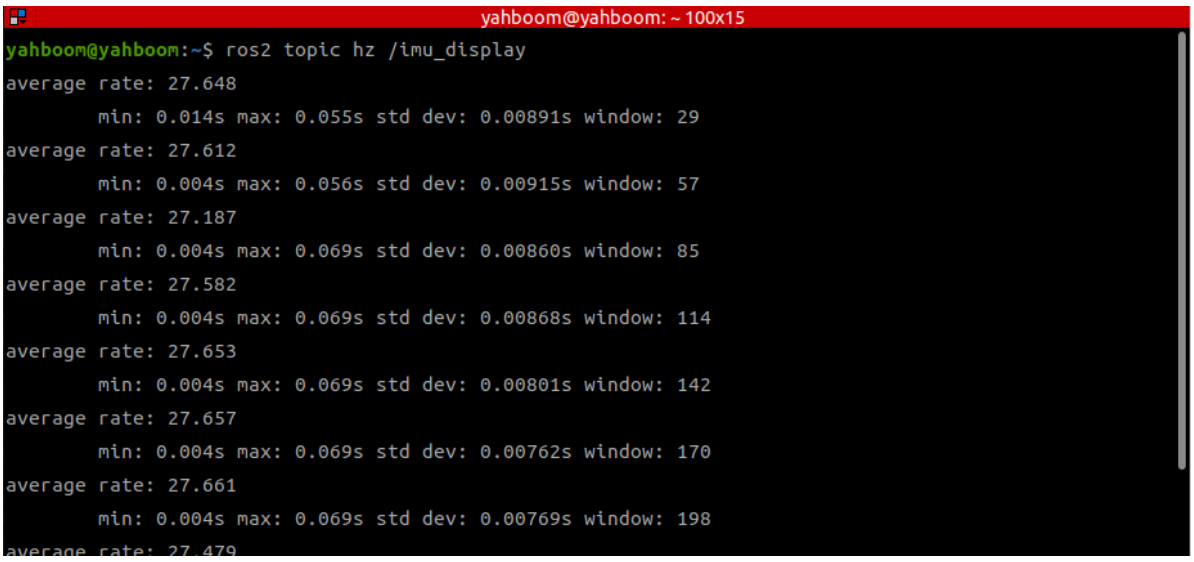
- View all topics

```
ros2 topic list
```



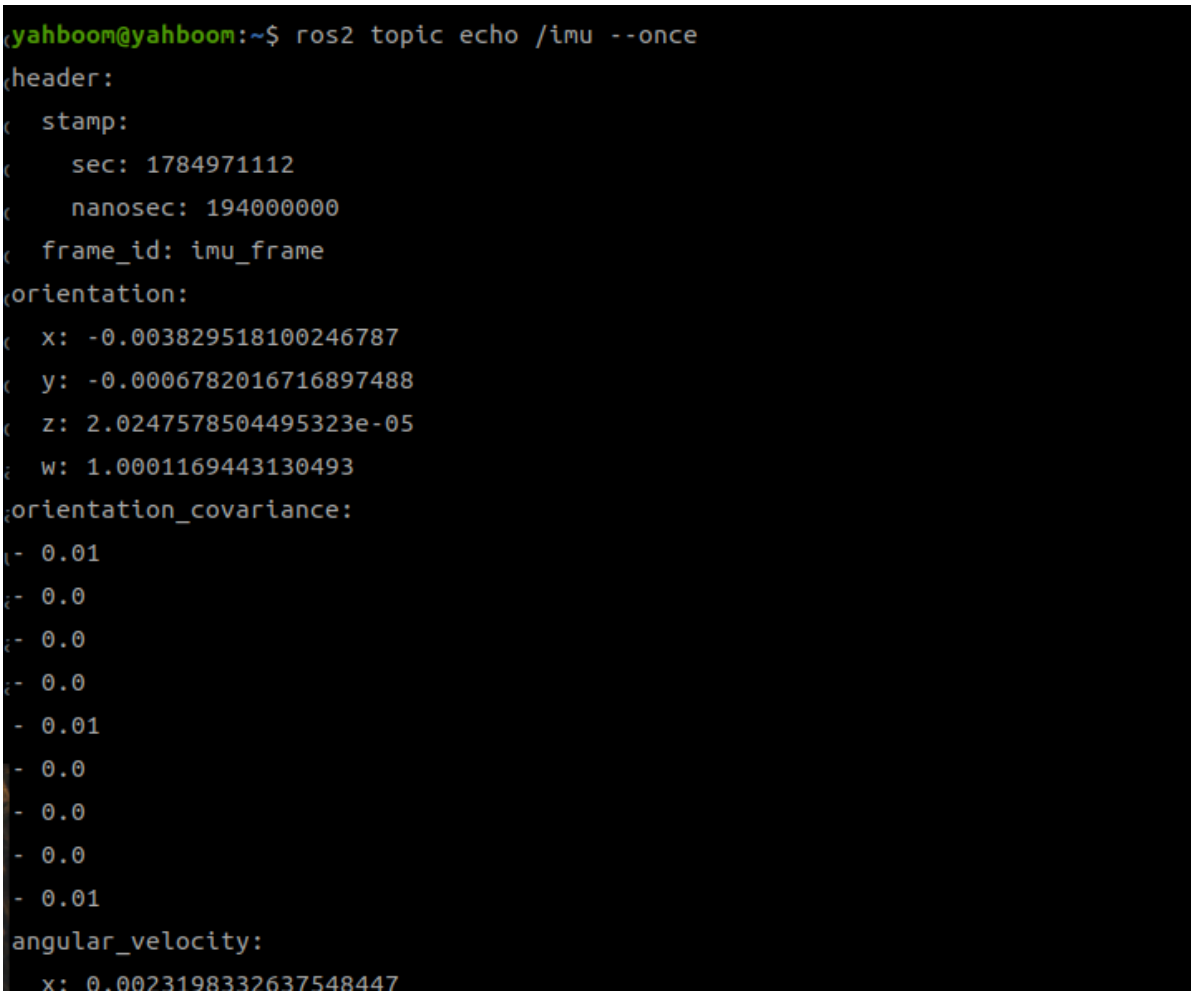
- View the publishing rate of a specified topic:

```
ros2 topic hz /imu
```



- View the data content of a specified topic

```
ros2 topic echo /imu --once
```

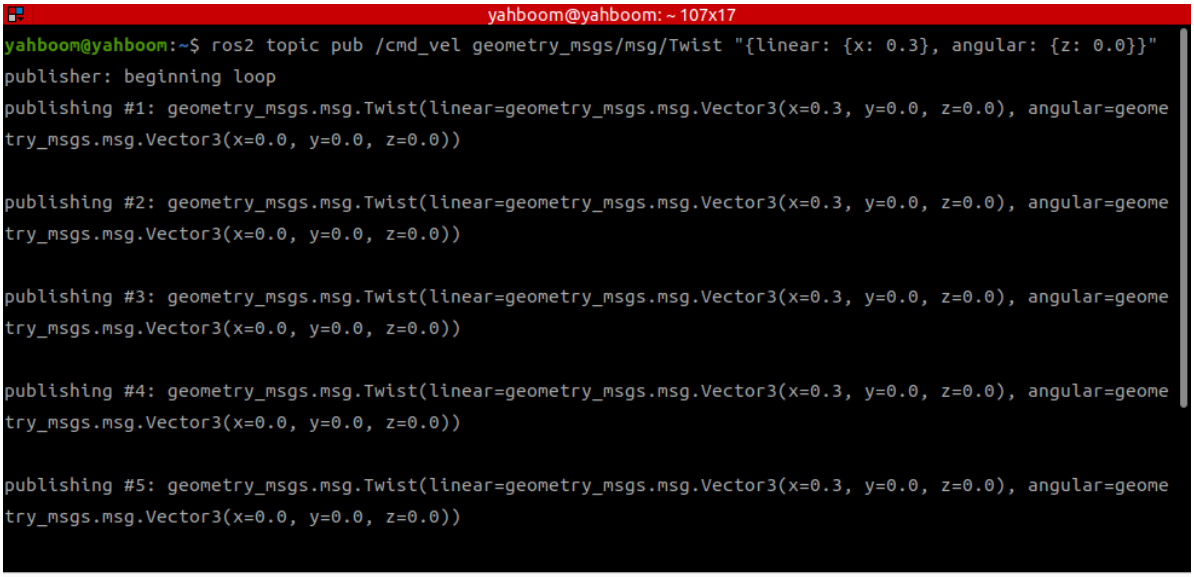


> [!TIP]
>
> The command format for viewing the data content, rate, and other information of other topics is the same; just replace the corresponding topic name

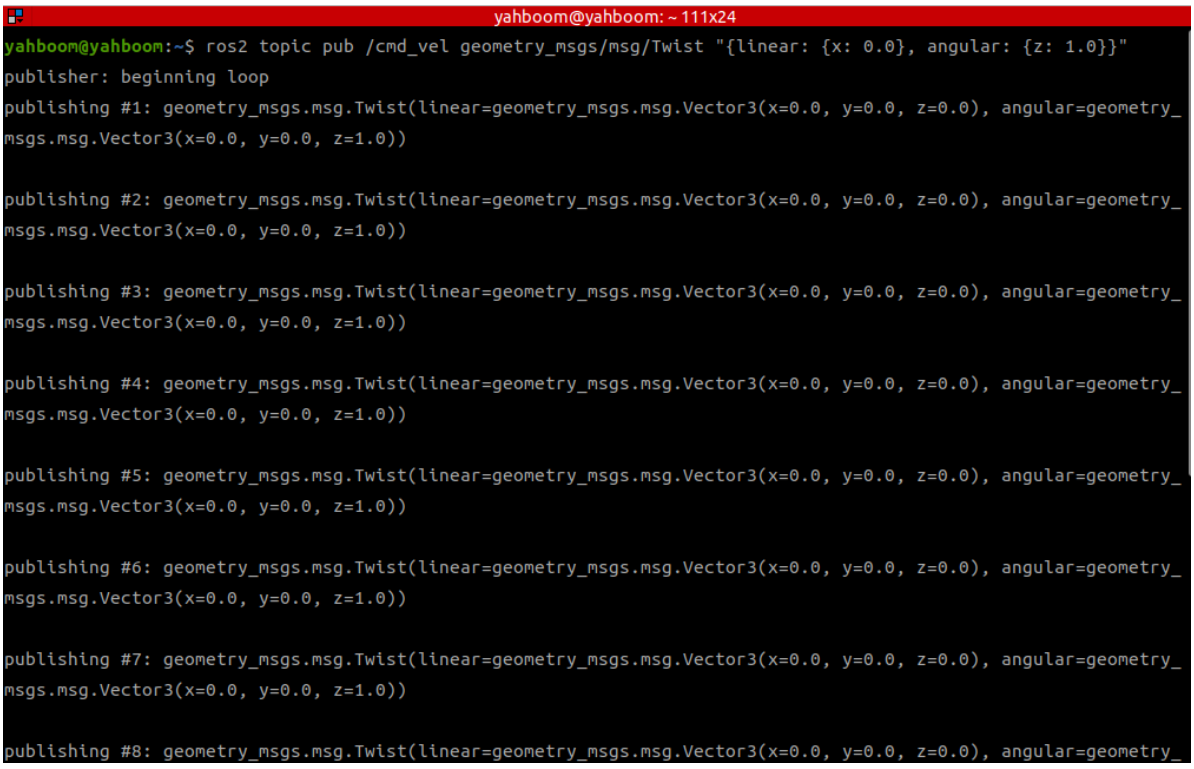
3.2 Controlling the Robot Chassis through Topics

- Publish velocity commands to `/cmd_vel` to control the car movement:

```
# Move forward at 0.3 m/s on the x-axis
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"
```




```
# Turn left in place at 1.0 rad/s
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{angular: {z: 1.0}}"
```



- Stop the car movement

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

3.3 Controlling the Buzzer On/Off through Topics

```
# Turn on the buzzer
ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 1}" --once
# Turn off the buzzer
ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 0}" --once
```

```
yahboom@yahboom:~$ ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 1}" --once
publisher: beginning loop
publishing #1: std_msgs.msg.UInt16(data=1)

yahboom@yahboom:~$ ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 0}" --once
publisher: beginning loop
publishing #1: std_msgs.msg.UInt16(data=0)
```

3.4 Controlling the Servo Gimbal Angle through Topics

> [!NOTE]
>
> x:0.0 ,y: -45.0 is the neutral position of the servo gimbal: the camera faces directly forward

```
# Control the servo gimbal to the specified angle
ros2 topic pub /cmd_ptz_angle geometry_msgs/msg/Vector3 "{x: 10.0, y: 15.0}" --
once
# Return the servo gimbal to the neutral position
ros2 topic pub /cmd_ptz_angle geometry_msgs/msg/Vector3 "{x: 0.0, y: -45.0}" --
once
```

3.5 Controlling Movement through Services

- Use the action service to move or rotate the chassis by a precise distance (based on odometry closed-loop control)

> [!NOTE]
>
> Parameter description:
>
> direction: movement direction; distance: movement distance; angle: rotation angle

- Usage examples

```
# Move forward 0.15 meters
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'forward', distance: 0.15, angle: 0.0}"
# Move backward 0.3 meters
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'backward', distance: 0.3, angle: 0.0}"
```

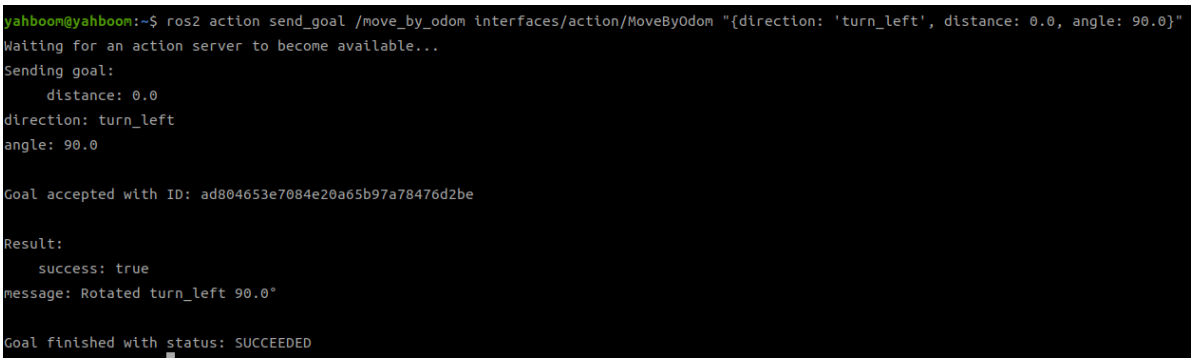
```
yahboom@yahboom:~$ ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction: 'forward', distance: 0.15, angle: 0.0}"
Waiting for an action server to become available...
Sending goal:
    distance: 0.15
direction: forward
angle: 0.0

Goal accepted with ID: 42c79e994a41410cb5f2a1f95add6886

Result:
   success: true
message: Moved forward 0.130m

Goal finished with status: SUCCEEDED
```

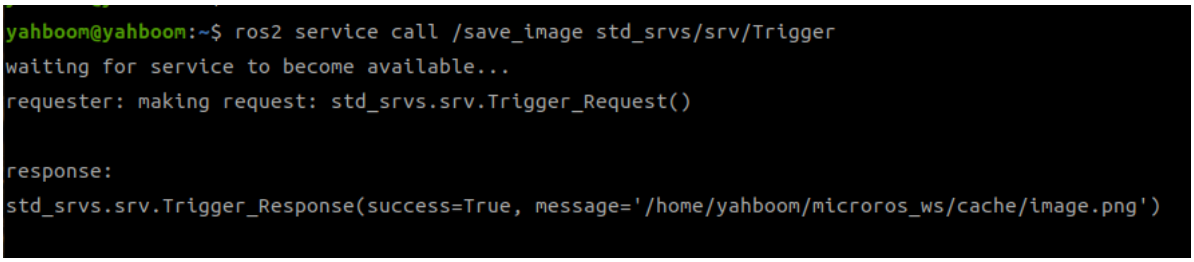
```
# Turn left 90 degrees
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'turn_left', distance: 0.0, angle: 90.0}"
# Turn right 45 degrees
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'turn_right', distance: 0.0, angle: 45.0}"
```



3.6 Saving Camera Images through a Service

- Save the latest image from the camera's video stream

```
ros2 service call /save_image std_srvs/srv/Trigger
```



- Default image save path: `$HOME/microros_ws/cache/image.png`. You can view it at the corresponding path in the virtual machine